

HTTP

Notes open for creative commons use by CryptoLain at <https://github.com/cryptolain> and klabweb@tuta.io

Learning Sources

<https://code.tutsplus.com/tutorials/http-the-protocol-every-web-developer-must-know-part-1--net-31177>

Intro to HTTP

- Hypertext Transfer Protocol
- Primary network protocol for communication between browsers and web servers
- Application layer protocol, usually over TCP/IP port 80
- Stateless - does not keep state between different message exchanges
- Response/request pair - how communication occurs. Client sends request. Server responds.

URLs

- Example <http://www.domain.com:1234/path/to/resource?a=b&x=y>
- Consists of
 - a) protocol - http
 - b) host – domain.com
 - c) port – 1234
 - d) /path/to/resource - resource path
 - e) query - ?a=b&x=y
- Default port is 80, but if want to specify alt, do as above

Verbs

- The action (request) that should be performed.

Main

- a) GET - fetch and existing resource. URL contains info on what is requested.
- b) POST - Create a new resource. Typically carries payload for data for new resource.
- c) PUT - Update an existing resource. Payload may contain update data.
- d) DELETE

Lesser Used

- e) HEAD - Get server headers for particular resource. Similar to GET, but only returns metadata for.
- f) TRACE - Retrieves hops request takes for round trip on server. Diagnostic use.
- g) OPTIONS - Retrieves server capabilities.

Status Codes

- Server responds to client requests with status codes and message payloads
- a) 1xxx: Informational messages
- b) 2xx: successful. Most common 200 OK. GET resource sent in message body.
- c) 3xx: redirection. Resource requested has been moved, etc..

- d) 4xx: client error. Client error. Ex. 404 Not Found, 403 Forbidden
- 3) 5xx: server error. Ex. 500 Internal Server Error, 503 Service Unavailable

Message Formats

-Requests and responses have following structure:

```
<start-line>
  *(<message-header>)
  CRLF
  [<message-body>]

Request-Line | Status-Line
Header-Name ':' Field-Value
```

-Request Example:

```
GET /articles/http-basics HTTP/1.1
Host: www.articles.com
Connection: keep-alive
Cache-Control: no-cache
Pragma: no-cache
Accept: text/html,application/xhtml+xml,application/xml;
```

-Response Example:

```
HTTP/1.1 200 OK
Date: Sat, 09 Oct 2010 14:28:02 GMT
Server: Apache
Last-Modified: Tue, 01 Dec 2009 20:18:22 GMT
ETag: "51142bc1-7449-479b075b2891b"
Accept-Ranges: bytes
Content-Length: 29769
Content-Type: text/html
```

<!DOCTYPE html... (here comes the 29769 bytes of the requested web page)

Headers

-In form *some-header = value*

a) General headers - syntax:

-ex. *Date, Transfer-Encoding*

b) Entity headers - provide meta-info content

-ex. *Content-Length, Content-Location, Content-Type, Last-Modified, Expires*

c) Request headers - headers that add detail to request

-ex. *Accept-Language, From, If-Unmodified-Since*

-*Host: somehost.com* - mandatory header for HTTP/1.1

Request Format

-Request line in format *someMethod /get/this HTTP/version*

-ex. GET /index.html HTTP/1.1

Response Format

-Format: *HTTP/version ### MESSAGE*

-ex. *HTTP/1.1 200 OK*

Response Headers

-ex. *Age, Retry-After, Server*

Inspector

-Can view http messages in Firefox via *Network* tool in Inspector